Semana 2: Python para Ciencia de Datos

Listas, Tuplas y Automatización con Bucles

Prof. Yazmin Rubiela Cote

Universidad Industrial de Santander

Matemáticas Computacionales

Hoja de Ruta de la Semana

- **1. Almacenar Datos:** Listas (Mutables) vs. Tuplas (Inmutables).
- **2. El Arte del Slicing:** Acceder a subconjuntos de datos.
- **3. Automatización:** Bucles `for` y `while`.
- **4. Reto** Análisis de precios de menú.

1. Tipos de Datos: Los Átomos de Python

Antes de correr, debemos saber de qué está hecho el suelo. En Python, todo es un objeto con un **tipo**.

Números

- `int`: Enteros (Sin límite de tamaño).
- `float`: Decimales (Precisión IEEE 754).

```
x = 10        # int
y = 3.1416    # float
z = 1.5e2     # Notaci n cient fica
    (150.0)
```

Booleanos (Lógica)

- Solo dos valores: `True` y `False`.
- Importante: ¡La primera letra es Mayúscula!

```
es_estudiante = True
aprobado = False
resultado = (10 > 5) # Devuelve True
```

Verificar Tipos (La función type)

Como científicos de datos, a veces los números vienen como texto desde un archivo CSV. Debemos detectar esto.

```
a = 50
b = "50"

print(type(a)) # <class 'int'>
print(type(b)) # <class 'str'>  <-  Cuidado  aqu !

# Conversi n de Tipos (Casting)
c = int(b) + a # Ahora s  suma 100

```

2. Manipulación de Texto (Strings)

Un String es técnicamente una **Lista Inmutable de Caracteres**. .

Escape Characters (Caracteres de Escape)

- \n: Salto de línea (Enter).
- \t: Tabulación.
- \': Comilla simple dentro de un string.

```
print("Hola\nMundo")
# Imprime:
# Hola
# Mundo

ruta = r"C:\Users\Nombre" # Raw String (Ignora los escapes)

```

Métodos Vitales para Data Science

Limpiar datos sucios es el 80 % del trabajo.

```python
texto = "   Python es Genial   "

# 1. Limpiar espacios (Trim)
print(texto.strip())  # "Python es Genial"

# 2. Dividir (String -> Lista)
palabras = texto.strip().split(" ")
# ['Python', 'es', 'Genial']

# 3. Unir (Lista -> String)
nueva_frase = "-".join(palabras)
# "Python-es-Genial"

```

f-Strings: El estándar moderno

Olviden el uso de + para concatenar o %s. Usen **f-strings**.

```
nombre = "Yazmin"
edad = 25
curso = "Matem ticas"

# Forma Antigua (No recomendada)
print("Hola " + nombre + " tienes " + str(edad) + " a os")

# Forma Pythonica (f-string)
print(f"Hola {nombre}, tienes {edad} a os.  xito  en {curso}.")

```

1. Listas vs. Tuplas: ¿Dónde guardo mis datos?

Listas [ ]

- Son **Mutables** (se pueden cambiar).
- Ideales para datos que crecen o cambian (ej. usuarios, mediciones).

```
planetas = ["Tierra", "Marte"]
planetas.append("Jupiter")
planetas[0] = "La Tierra"
# Funciona perfecto

```

Tuplas ( )

- Son **Inmutables** (fijas).
- Ideales para constantes (ej. coordenadas, dimensiones).

```
dimensiones = (1920, 1080)
# dimensiones[0] = 400
#   ERROR ! No se puede cambiar

```

2. Indexación: El mapa del tesoro

Python cuenta desde **CERO**. Y tiene un superpoder: **Índices Negativos**.

```
#                    0          1          2          3
frutas = ["Manzana", "Banana", "Cereza", "Durazno"]
#                   -4         -3         -2         -1

```

- `frutas[0]` → "Manzana" (El primero)
- `frutas[-1]` → "Durazno" (El último, ¡muy útil!)

El Arte del Slicing (Rebanar)

Sintaxis: `lista[inicio : fin : paso]` ¡OJO! El límite superior (fin) NO se incluye.

```python
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Los primeros 3
print(numeros[0:3])  # Salida: [0, 1, 2]

# Desde el 5 hasta el final
print(numeros[5:])   # Salida: [5, 6, 7, 8, 9]

# Invertir una lista (Truco Pro)
print(numeros[::-1]) # Salida: [9, 8, ... 0]

```

3. Bucle FOR: "Para cada elemento..."

Se usa cuando sabemos **cuántas veces** queremos repetir algo o para recorrer una colección.

```
invitados = ["Ana", "Bob", "Clara"]

# Iterar sobre la lista
for persona in invitados:
    print("Hola " + persona)

# Iterar sobre una secuencia num rica (range)
for i in range(5):
    print("Repetici n  n mero:", i)

```

Bucle WHILE: "Mientras ocurra que..."

Se usa cuando **NO sabemos** cuándo terminará, depende de una condición lógica.

```
tanque = 0
capacidad_max = 5

while tanque < capacidad_max:
    print(f"Llenando... Nivel actual: {tanque} galones")
    tanque = tanque + 1

print("  Tanque   lleno!")

```

Listas: Operaciones Básicas (Métodos)

Las listas en Python son **mutables** y ya traen sus propias herramientas integradas para modificarse fácilmente.

```python
planetas = ["Tierra", "Marte", "Jupiter", "Tierra"]

# 1. A adir un elemento (.append) -> Lo pone al final
planetas.append("Saturno")
# ['Tierra', 'Marte', 'Jupiter', 'Tierra', 'Saturno']

# 2. Borrar un elemento (.remove) -> Borra la primera aparici n
planetas.remove("Marte")
# ['Tierra', 'Jupiter', 'Tierra', 'Saturno']

# 3. Contar elementos (.count)
numero_tierras = planetas.count("Tierra")
print(f"La Tierra aparece {numero_tierras} veces.")

# Extra: Ordenar la lista (.sort)
planetas.sort()
```

Ejercicio 1: El Analista de Precios

Problema: Tienes una lista de precios de un menú, pero hubo inflación del 10 %.

Reto

Crea un bucle que recorra la lista, aplique el aumento y guarde los nuevos precios.

Ejercicio 1: El Analista de Precios

Problema: Tienes una lista de precios de un menú, pero hubo inflación del 10 %.

Reto

Crea un bucle que recorra la lista, aplique el aumento y guarde los nuevos precios.

```python
precios = [2000, 5500, 1200, 8000]
nuevos_precios = []

for precio in precios:
    aumento = precio * 1.10  # Aumentar 10%
    nuevos_precios.append(aumento)

print(nuevos_precios)
```

Ejercicio 2: Limpieza de Datos (Slicing)

Problema: Un sensor envió datos basura al inicio y al final de la transmisión.

```
# Datos recibidos: [Error, Error, 15.5, 16.2, 14.8, 15.9, Error]
datos = ["Error", "Error", 15.5, 16.2, 14.8, 15.9, "Error"]

```

Misión: Usa Slicing para extraer solo los datos válidos (los números del centro).

Ejercicio 2: Limpieza de Datos (Slicing)

Problema: Un sensor envió datos basura al inicio y al final de la transmisión.

```
# Datos recibidos: [Error, Error, 15.5, 16.2, 14.8, 15.9, Error]
datos = ["Error", "Error", 15.5, 16.2, 14.8, 15.9, "Error"]

```

Misión: Usa Slicing para extraer solo los datos válidos (los números del centro).

Solución

```
datos_limpios = datos[2:-1]
# Desde el  ndice  2 hasta el pen ltimo (sin incluir el  ltimo  error)
print(datos_limpios)

```

Ejercicio 3: Limpieza de Datos de un Sensor

Contexto: Un sensor de temperatura en un laboratorio registra datos cada hora. Sin embargo, el sensor tuvo un fallo eléctrico y registró un valor imposible. Además, acaba de llegar una nueva lectura.

Datos Iniciales

```
temperaturas = [22.5, 23.1, 99.9, 21.8, 22.5, 24.0]
```

Tu Misión (Usando solo métodos de listas):

1. Acaba de llegar la lectura `23.5`. Añádela a la lista.
2. El valor `99.9` es un error del sensor. Bórralo de la lista.
3. El valor `22.5` parece ser la moda. Cuenta cuántas veces aparece.
4. Ordena la lista de menor a mayor para ver fácilmente la temperatura mínima y máxima.

Solución al Reto

```python
temperaturas = [22.5, 23.1, 99.9, 21.8, 22.5, 24.0]

# 1. A adir la nueva lectura
temperaturas.append(23.5)

# 2. Eliminar el error
temperaturas.remove(99.9)

# 3. Contar la moda
frecuencia = temperaturas.count(22.5)
print(f"El valor 22.5 aparece {frecuencia} veces.")

# 4. Ordenar para an lisis
temperaturas.sort()
print(f"Datos limpios y ordenados: {temperaturas}")
print(f"Min: {temperaturas[0]} | Max: {temperaturas[-1]}")
```

1. List Comprehensions: Elegancia Matemática

Como matemáticos, preferimos definir conjuntos por comprensión: $S = \{x^2 | x \in N, x < 10\}$. Python permite hacer exactamente lo mismo. Es más rápido y legible.

1. List Comprehensions: Elegancia Matemática

Como matemáticos, preferimos definir conjuntos por comprensión: $S = \{x^2 | x \in N, x < 10\}$. Python permite hacer exactamente lo mismo. Es más rápido y legible.

Forma Clásica (C/Java style)

```
cuadrados = []
for x in range(10):
    cuadrados.append(x**2)

```

1. List Comprehensions: Elegancia Matemática

Como matemáticos, preferimos definir conjuntos por comprensión: $S = \{x^2 | x \in N, x < 10\}$. Python permite hacer exactamente lo mismo. Es más rápido y legible.

Forma Clásica (C/Java style)

```python
cuadrados = []
for x in range(10):
    cuadrados.append(x**2)

```

Forma Pythonica

```python
# [expresion FOR elemento IN iterable]
cuadrados = [x**2 for x in range(10)]

# Con condicional: Solo pares
pares = [x for x in range(10) if x % 2 ==
    0]

```

Ejercicio Práctico: Filtrado Múltiple

El Problema Matemático: Queremos construir el conjunto de todos los números del 1 al 30 que sean simultáneamente **pares** y **múltiplos de 3**.

$$S = \{x \in \mathbb{N} \mid 1 \leq x \leq 30,\ x \text{ mód } 2 = 0 \text{ y } x \text{ mód } 3 = 0\}$$

Reto Computacional

Traduce este conjunto a una **List Comprehension** en Python en una sola línea de código.

Ejercicio Práctico: Filtrado Múltiple

El Problema Matemático: Queremos construir el conjunto de todos los números del 1 al 30 que sean simultáneamente **pares** y **múltiplos de 3**.

$$S = \{x \in \mathbb{N} \mid 1 \leq x \leq 30,\ x \text{ mód } 2 = 0 \text{ y } x \text{ mód } 3 = 0\}$$

Reto Computacional

Traduce este conjunto a una **List Comprehension** en Python en una sola línea de código.

Solución:

```python
# Usamos 'and' para encadenar m ltiples filtros
S = [x for x in range(1, 31) if x % 2 == 0 and x % 3 == 0]

print(S)
# Salida: [6, 12, 18, 24, 30]

```

Ejercicio Práctico: Filtrado Múltiple

El Problema Matemático: Queremos construir el conjunto de todos los números del 1 al 30 que sean simultáneamente **pares** y **múltiplos de 3**.

$$S = \{x \in \mathbb{N} \mid 1 \leq x \leq 30,\ x \text{ mód } 2 = 0 \text{ y } x \text{ mód } 3 = 0\}$$

Reto Computacional

Traduce este conjunto a una **List Comprehension** en Python en una sola línea de código.

Solución:

```python
# Usamos 'and' para encadenar m ltiples filtros
S = [x for x in range(1, 31) if x % 2 == 0 and x % 3 == 0]

print(S)
# Salida: [6, 12, 18, 24, 30]

```

Diccionarios: Estructuras Clave-Valor

A diferencia de las listas (donde buscamos por la posición 0, 1, 2...), en los diccionarios buscamos los datos mediante una **Clave (Key)** que está asociada a un **Valor (Value)**.

- Se definen usando llaves $\{\}$.
- Son increíblemente rápidos para buscar información.
- Las claves deben ser únicas (no se pueden repetir).

```python
# Creando un diccionario de un estudiante
estudiante = {
    "nombre": "Ana",
    "carrera": "Matem ticas",
    "semestre": 5
}

# Acceder a un valor usando su clave
print(estudiante["carrera"])  # Salida: Matem ticas

# Agregar un nuevo par Clave-Valor
estudiante["promedio"] = 4.5
```

2. Diccionarios (Hash Maps)

Ejemplo de diccionario

```python
# Base de datos de un estudiante
estudiante = {
    "nombre": "Ana",
    "edad": 22,
    "notas": [3.5, 4.0, 4.8]
}

# Acceder y Modificar (O(1) - Instant neo)
print(estudiante["nombre"])      # 'Ana'
estudiante["promedio"] = sum(estudiante["notas"]) / 3

```

3. Iteración Pro: Enumerate y Zip

Regla de Oro: Si estás haciendo `range(len(lista))`, probablemente hay una forma mejor.

A. Enumerate (Índice y Valor al tiempo)

```
nombres = ["Ana", "Bob", "Cale"]
for i, nombre in enumerate(nombres):
    print(f"Estudiante #{i+1}: {nombre}")

```

B. Zip (Iterar dos listas en paralelo)

```
nombres = ["Ana", "Bob"]
notas = [4.5, 3.2]
for n, p in zip(nombres, notas):
    print(f"{n} sac  {p}")

```

Reto 1: Procesamiento de Lenguaje (NLP Básico)

Contexto: Tienes tweets sucios. Necesitas limpiarlos, ponerlos en minúsculas y quitar los cortos.

```
tweets = ["Hola MUNDO!", "  Python es GENIAL ", "ok", "Data Science"]

```

Misión: Hazlo en UNA sola línea usando List Comprehension.

- Quitar espacios extra (`.strip()`).
- Convertir a minúsculas (`.lower()`).
- Solo si la longitud es mayor a 3.

Reto 1: Procesamiento de Lenguaje (NLP Básico)

Contexto: Tienes tweets sucios. Necesitas limpiarlos, ponerlos en minúsculas y quitar los cortos.

```
tweets = ["Hola MUNDO!", "  Python es GENIAL ", "ok", "Data Science"]

```

Misión: Hazlo en UNA sola línea usando List Comprehension.

- Quitar espacios extra (`.strip()`).
- Convertir a minúsculas (`.lower()`).
- Solo si la longitud es mayor a 3.

```
clean = [t.strip().lower() for t in tweets if len(t.strip()) > 3]
# Resultado: ['hola mundo!', 'python es genial', 'data science']

```

Reto 2: Frecuencia de Palabras (Histograma)

Problema: Cuenta cuántas veces aparece cada número en la lista. *Pista: Usa un diccionario.*

Reto 2: Frecuencia de Palabras (Histograma)

Problema: Cuenta cuántas veces aparece cada número en la lista. *Pista: Usa un diccionario.*

```python
datos = [1, 3, 2, 1, 5, 3, 1, 5]
conteos = {}

for num in datos:
    if num in conteos:
        conteos[num] += 1
    else:
        conteos[num] = 1

print(conteos)
# Resultado: {1: 3, 3: 2, 2: 1, 5: 2}
```

Resumen para la Próxima Clase

Hemos cubierto la base del procesamiento de datos en Python:

- **Tipos:** Sabemos distinguir entre `int`, `float` y `str`.
- **Lógica:** Usamos Booleanos para controlar el flujo.
- **Texto:** Sabemos limpiar y formatear strings (‘split‘, ‘join‘, ‘strip‘).
- **Simulación:** Usamos listas y random para modelar problemas matemáticos.